

Project Plan: AWS Deployment Service

Integration with Build Orchestrator

Introduction

This project involves creating a new **AWS integration web service** (an "HTML service") that works in tandem with an existing **build orchestrator**. The goal is to automate the setup and teardown of a complete AWS deployment environment on-demand. In essence, this new service will provide a RESTful API offering low-level AWS operations (primitives), while the build orchestrator will sequence these operations to provision or destroy infrastructure. This approach is similar in spirit to Infrastructure-as-Code tools like Terraform and CloudFormation (which allow you to define and manage cloud infrastructure programmatically ¹ ²), but with a custom implementation tailored to our environment. The key difference is that our build service will handle orchestration using fine-grained API calls, with a strong emphasis on **security (no direct credential exposure)** and flexibility in deployment targets (any AWS region or environment).

Background and Context

The existing build service already acts as an **orchestrator** for deployments. For example, a GitHub integration service is in place where the build service can trigger actions on GitHub without holding any Git credentials in plain text. In that design, the build orchestrator obtains a public key from the GitHub service and uses it to encrypt a Personal Access Token (PAT). The encrypted token is then forwarded to the GitHub service, which alone can decrypt it and perform the needed GitHub API calls. The build service itself never sees the plaintext credentials. This secure pattern has proven effective in practice and will be re-used for AWS.

Currently, many AWS setup tasks (such as configuring serverless components and other infrastructure) have been done manually. The aim is to **automate these manual steps**. The build service today already has some capabilities analogous to Terraform/CloudFormation – it can coordinate multi-step deployment processes – but it relies on external services for carrying out specific tasks. By introducing a dedicated AWS integration service, we centralize AWS-related operations behind a clean API. The build orchestrator will invoke this API to provision AWS resources in the correct order, effectively enabling **one-click (or one-command) environment provisioning**.

Goals and Objectives

- **Full Infrastructure Automation:** Enable spinning up an entire application environment from scratch in AWS, and tearing it down, on demand. This includes all necessary components (compute, storage, networking, serverless functions, etc.) for a functional deployment. The solution should support deploying to any AWS region or account as needed by specifying parameters.
- **Modularity via Primitives:** Expose fine-grained AWS operations (primitives) as individual API endpoints. Each primitive corresponds to creating or checking a specific AWS resource (or performing a specific AWS action). This allows the build orchestrator to flexibly compose complex workflows. Rather than a monolithic "deploy everything" script, the orchestrator will call a series

of small, focused actions like “create S3 bucket”, “launch EC2 instance”, “create Lambda function”, etc.

- **Orchestrator-Driven Flow:** Keep orchestration logic in the build service. The new AWS service will **not** itself orchestrate multi-step processes; it will not, for example, automatically wait for one resource to be ready before creating another. Instead, it simply executes the requested primitive action and returns the result. The build orchestrator is responsible for calling the right primitives in sequence and handling dependencies between resources. This separation of concerns makes the system more flexible and testable.
- **Security:** Preserve the security model used in the GitHub integration – the build orchestrator should not handle sensitive AWS credentials directly. All AWS API calls will require credentials that are passed securely (encrypted) to the AWS service. The AWS service will decrypt and use them internally. This minimizes the exposure of secrets. Additionally, the AWS credentials used should ideally have fine-tuned IAM permissions, limiting the actions to only those needed for the deployment tasks.
- **Repeatability and Consistency:** Ensure that the same series of actions can reliably produce a working environment. By automating the steps, we eliminate configuration drift and human error. The process should be consistent every time. We should also be able to destroy an environment completely to avoid orphaned resources, ensuring cost control and a clean state for re-deployment.
- **Multi-Mode Deployment Support:** The system should handle different “modes” or deployment targets. For example, parts of the application might run in a **serverless mode** (AWS Lambda functions) and other parts in a **serverful mode** (EC2 instances behind an Auto Scaling Group). The build orchestrator might decide which primitives to call based on the target deployment mode (for instance, in a purely serverless environment it might skip EC2/ASG calls). The AWS service should therefore provide primitives covering both scenarios, but not enforce any particular combination – that logic lives in the orchestrator.

Proposed Architecture Overview

Components & Roles:

- **Build Orchestrator Service (Existing):** Acts as the brain of the deployment process. It doesn’t execute cloud actions directly but knows the desired deployment workflow. It will call the AWS service’s API endpoints in the appropriate order to achieve the deployment. It also manages obtaining and encrypting credentials. Think of this as analogous to a controller or the Terraform engine – it decides *what* to do and *when*, but delegates the *how* to the AWS service.
- **AWS Integration Service (To Be Developed):** A new web service (referred to as the HTML/HTTP service) providing REST API endpoints for AWS operations. This service will be built on top of our existing **OSBot-AWS** library, which offers simplified, Pythonic wrappers around AWS SDK (boto3) calls ³ ⁴. The service will essentially expose functions from OSBot-AWS as HTTP endpoints. Each endpoint corresponds to a primitive action (e.g., create an S3 bucket, launch an EC2 instance, etc.). The service will handle authentication (decrypting credentials), invoke OSBot-AWS functions to interact with AWS, and return results/status. It will not contain orchestration logic across multiple endpoints – each call is a single, atomic AWS operation.
- **OSBot-AWS Library (Existing):** A Python library in the OWASP Security Bot project that provides high-level abstractions over AWS services ³. By using OSBot-AWS under the hood, the new service benefits from **simplified APIs and type-safe wrappers** for AWS, reducing complexity and boilerplate in our implementation ⁴. For example, OSBot-AWS can create resources with simple method calls (e.g., one call to create an S3 bucket) rather than dealing with low-level boto3 calls and error handling each time. This should speed up development of the AWS service and ensure reliability.
- **AWS Cloud (Target Environment):** The actual AWS account(s) and regions where infrastructure will be created. The AWS service will perform actions against these accounts using provided credentials. We

must consider that the AWS service might deploy to multiple accounts or regions, depending on the provided credentials and parameters in each API call. Thus, the design should not hard-code any single account details; everything is driven by input parameters.

Credential Handling Workflow:

A crucial aspect of the architecture is how AWS credentials are handled securely between the build orchestrator and the AWS service. The following steps outline the proposed workflow for credential exchange:

1. **Public Key Retrieval:** When the build orchestrator needs to perform AWS actions, it first requests the AWS service's public key. We will provide an endpoint on the AWS service (e.g., `GET /aws/public-key`) that returns its public RSA key (or similar). This key pair is generated and kept within the AWS service (the private key never leaves the service).
2. **Credential Encryption:** The build orchestrator (or a secure client in the pipeline) will take the AWS credentials (Access Key ID, Secret Access Key, and session token if applicable) and encrypt them using the public key. By doing this, the credentials become an opaque blob that only the AWS service can decrypt with its private key.
3. **API Call with Encrypted Credentials:** The build orchestrator then calls the desired AWS service endpoint (for example, the "create bucket" API) and includes the encrypted credentials in the request payload (along with other parameters needed for that action, like bucket name, region, etc.). Each API request must carry the credentials (encrypted) because the AWS service is stateless between requests and should not store credentials long-term. This also allows each call to potentially target different AWS accounts/regions if needed by providing different credentials.
4. **Decryption and AWS Action:** Upon receiving the request, the AWS service uses its private key to decrypt the credentials in memory. It then uses OSBot-AWS (which in turn uses boto3 under the hood) to assume an AWS session with those credentials. Once authenticated, the service executes the specific AWS API call (e.g., create the S3 bucket) using the OSBot-AWS library function.
5. **Result Handling:** The AWS service collects the result of the operation (e.g., success confirmation, resource ID/ARNs, or any error message from AWS if the operation failed). It returns a response to the build orchestrator with the outcome. No sensitive credential data is included in the response. After using the credentials for the call, the service should purge them from memory (ensuring they aren't logged or stored).
6. **Orchestration Continues:** The build orchestrator receives the result and proceeds to the next step in the deployment plan, calling further AWS service endpoints as needed. If any step fails (an endpoint returns an error), the orchestrator can decide to halt the process and perform any cleanup (for example, if bucket creation failed, perhaps abort the whole deployment). Because each action is separate, we have flexibility in handling partial failures (possibly implementing retries or conditional logic in the orchestrator).

This design ensures **secure credential forwarding** – the build orchestrator never needs to handle raw AWS secrets, aligning with the project's security objectives. It also means the AWS service can be deployed in a secure environment (possibly within the target AWS account or a management account) with its own key pair, and the build orchestrator can be run elsewhere (like a CI/CD runner or developer machine) without risking credential exposure. The trade-off is a bit more complexity in making sure the encryption/decryption mechanism is in place, but this is a known pattern we have used successfully for GitHub integration.

(Side note: In development or testing, we might allow a "direct mode" where credentials are passed in plaintext just for simplicity, but in production the encryption should be mandatory. We could make the

encryption step optional via config to ease local testing, though using dummy credentials might suffice as well.)

AWS Service API Primitives

The AWS integration service will expose a set of REST API endpoints, each corresponding to a fundamental AWS operation. These endpoints will likely be organized by resource type or AWS service. Below is a detailed list of the primitives we plan to implement, based on the voice memo requirements and the target infrastructure components:

- **S3 Bucket Management:**

- **Create S3 Bucket:** Endpoint to create an S3 bucket. Input parameters might include bucket name, region, and optional settings (like versioning, encryption settings if needed). The service will call OSBot-AWS to create the bucket (e.g., `s3.bucket_create(name, region)` using OSBot-AWS ⁵). It should handle the case where the bucket already exists (likely returning a specific response indicating existence or a conflict).
- **Check S3 Bucket:** Endpoint to verify a bucket's existence or retrieve its properties. This could simply attempt to `head` the bucket or list it to confirm it's present. Useful for the orchestrator to confirm that creation was successful (especially if eventual consistency is a concern) or to skip creation if the bucket is already present.
- **Delete (Free) S3 Bucket:** Endpoint to delete a bucket. Because S3 buckets must be empty before deletion, this action will first ensure all objects (and versions, if versioned) are removed (this is what we mean by "freeing" the bucket, i.e., emptying it), then delete the bucket itself. This will be used during teardown of an environment. It should handle errors like bucket not found or not empty (if some objects couldn't be deleted due to permissions).

- **EC2 and AMI Operations:**

- **Create AMI (Amazon Machine Image):** An endpoint to create an AMI, likely by taking a snapshot of a running EC2 instance or from an EBS snapshot. The context here is that to deploy our application on EC2 instances at scale (auto-scaling group), we might need a custom AMI that has our application pre-installed. This endpoint would accept parameters like a source instance ID (which has the application installed) and an AMI name, and it would call the AWS API to create an image from that instance. Alternatively, if we are using a predefined base AMI plus cloud-init scripts, this step might not be needed every deployment – but having it as an option is useful for baking images. The output would be the new AMI ID.
- **Launch EC2 Instance (Execute AMI):** Endpoint to launch an EC2 instance from a specified AMI. Input might include the AMI ID, instance type, security group, subnet, etc., and possibly user-data scripts. In the voice memo this was described as "execute AMI". Essentially, this means bringing an instance online using the given machine image. This could be used in two scenarios: (a) to create a one-off instance (maybe to run some setup or tests), or (b) to launch a baseline instance that might then be imaged (in combination with the above AMI creation). It might also be part of a non-auto-scaling deployment where a single instance is enough. The endpoint will return instance details (instance ID, IP address, etc.).
- **Execute Commands on EC2:** An endpoint to run commands on a running EC2 instance. This is likely to be implemented via AWS SSM (Systems Manager) Run Command or via an OSBot-AWS convenience method. The idea is to automate any configuration steps on the server that would normally be done manually. For example, after launching an instance, we might need to run configuration scripts or verify something on it. By having a primitive to execute commands, the

orchestrator can perform in-instance setup tasks (like installing application components, configuring the environment, or running smoke tests). Input would be the instance ID and the command or script to run (or perhaps a reference to a pre-script stored in S3 or parameter store). Output would include the command execution status and output logs. This capability is important for fully automated setup, especially if not everything is baked into the AMI.

- **Load Balancer and Auto Scaling:**

- **Create Target Group:** Endpoint to create a Target Group for an Application Load Balancer (ALB) or Network Load Balancer. A target group is required to register the EC2 instances (or ECS tasks) so that a load balancer can route traffic to them. Input parameters could include the target group name, the target port, protocol, VPC identifier, and health check settings. The service will call AWS to create the target group and return its ARN. This is typically used in combination with an ALB, which we might assume exists or create separately. (We might need a companion endpoint to **Create Load Balancer** if one isn't already in place. The voice memo didn't explicitly mention creating the ALB itself, but since target groups are part of load balancing, we should consider if the ALB is pre-existing or also needs to be created. For completeness, we could include an endpoint for creating an ALB + listener that uses the target group.)
- **Create Auto Scaling Group (ASG):** Endpoint to create an Auto Scaling Group that will manage a set of EC2 instances across multiple availability zones. Input likely includes: name of ASG, launch configuration or launch template (which includes the AMI ID or instance configuration to launch), the desired capacity, min/max size, the target group (so that the instances register to the load balancer), and scaling policies (if any). Creating an ASG might involve first creating a **Launch Template** (or Launch Configuration) that specifies the AMI, instance type, security groups, and startup scripts. We might expose an endpoint for creating a Launch Template as well, or accept the necessary parameters and have the AWS service internally create a launch template as part of ASG creation. For simplicity, we could bundle it: if an AMI ID is provided to the ASG creation endpoint along with other instance settings, the service can create a launch template on the fly and then create the ASG. The result will be the ASG identifier and confirmation that instances are launching. The orchestrator might then want to verify that the ASG has successfully launched the instances and that they become healthy in the target group.
- **Check/Verify ASG (and ALB):** We should have endpoints to check the status of the ASG – for example, retrieving the current number of instances and their health status. Also, if an ALB is involved, an endpoint to check that the load balancer has the new instances in service (perhaps by describing target health in the target group). These checks allow the orchestrator to wait until the application is actually up and running before signaling that deployment is complete.
- **Tear Down ASG/Load Balancer:** For completeness, endpoints to delete the auto scaling group (which would terminate the EC2 instances), delete the target group, and delete the load balancer if we created one. These would be used in the teardown sequence to dismantle the environment.

- **AWS Lambda Functions:**

- **Create Lambda Function:** Endpoint to create or update a Lambda function. Input parameters include function name, runtime, IAM execution role ARN, and the deployment package (code). We might provide the code via an S3 bucket reference (which is common for larger deployments – upload the package to S3 then point Lambda to it) or possibly via direct code if small. OSBot-AWS likely has utilities for creating Lambda functions easily (as shown in its documentation, it can set the S3 bucket and key for Lambda code and then call `update()` to create or update the function ⁶ ⁷). We will leverage that. The endpoint should create the function and also configure any necessary settings like environment variables for the function.

- **Configure Lambda Function URL:** The plan is to allow our Lambda functions to be invoked via public URLs (AWS Lambda Function URLs, a feature that allows HTTPS endpoints for lambdas without needing API Gateway). We need to ensure that when we create a Lambda function, we have the option to create a Function URL attached to it. This could be done in two ways: either the **Create Lambda** endpoint accepts a flag like `enable_url: true` and, after creating the function, the service calls AWS to create a Function URL and returns that URL; or we provide a separate endpoint specifically to create/enable the Function URL for an existing function. The voice memo suggests it as a property of the function creation ("lambda functions should have the properties of e.g. function URL and all that jazz"), which implies the first approach – incorporate it into the creation process for convenience. We will implement it such that if the request asks for a function URL, the AWS service will call the necessary AWS API to create a function URL binding and then include the URL in the response.
- **Check Lambda Function:** Endpoint to verify that a Lambda function exists and is ready. This can simply fetch the function configuration or attempt to invoke it with a test event to ensure it's working. The orchestrator might call this after creation to double-check or to retrieve attributes like the Function ARN or URL.
- **Delete Lambda Function:** Endpoint to delete a Lambda function (used during teardown). This will remove the function and its associated Function URL. If the function had any other resources (like log groups or IAM roles), those might persist unless cleaned up, but for now the focus is on removing the function so it no longer incurs cost or conflicts with future deployments of the same name.

Each of these endpoints will be documented with its HTTP method (likely `POST` for create actions, `GET` for checks, and `DELETE` for deletions, or `POST` with an action field if we unify under one path), required input parameters, and the response format. Collectively, they form a **functionally-driven API** for AWS operations. The build orchestrator can pick and choose which to call and in what order, giving us a lot of control – akin to building our own tailored CloudFormation, but imperatively.

A quick example of how some endpoints might look:

- `POST /aws/s3/bucket/create` – body: `{ name: "my-bucket", region: "us-east-1", credentials: "<encrypted_blob>" }`
- `GET /aws/s3/bucket/check?name=my-bucket®ion=us-east-1` – (credentials likely in headers or a secure store reference) returns 200 if exists.
- `POST /aws/ec2/ami/create` – body: `{ instance_id: "i-01234", name: "my-ami-20231101", credentials: "<encrypted>" }` returns new AMI ID.
- `POST /aws/ec2/instance/launch` – body: `{ ami_id: "ami-05678", instance_type: "t3.medium", security_group: "...", subnet: "...", user_data: "...", credentials: "..." }` returns instance ID/IP.
- `POST /aws/ec2/instance/run-command` – body: `{ instance_id: "i-07890", command: "bash install.sh", credentials: "..." }` returns command output.
- `POST /aws/autoscaling/group/create` – body with all needed params (could be complex, possibly we break it down into launching template and ASG separately).
- `GET /aws/autoscaling/group/status?name=XYZ` – returns JSON with current size, instance IDs, health states.
- `POST /aws/lambda/create` – body: `{ function_name: "my-func", runtime: "python3.9", role: "arn:aws:iam:...:role/xyz", s3_bucket: "deploy-packages", s3_key: "myfunc.zip", env: {...}, enable_url: true, credentials: "..." }` – creates the function and returns function ARN and URL if created.
- `GET /aws/lambda/check?function_name=my-func` – returns function configuration or a simple status.

(The exact API design will be finalized in the specification document, but the above gives an idea of the granularity and shape of the calls.)

By exposing these granular operations, we allow not only the orchestrated full deployment but also the possibility to run or test individual actions in isolation (for example, an engineer could call the API to just create a bucket or just deploy a lambda without doing the whole environment). This can be useful for debugging or for reusing the service for other automation tasks in the future. It's essentially an internal AWS operations API for our team.

Orchestration Flow in the Build Service

With the primitives in place, the build orchestrator can implement the **deployment logic** as a series of API calls. This logic can be thought of as a script or workflow, and it can vary depending on the type of deployment (e.g., full stack with EC2, or serverless-only, etc.). Below is a detailed example of how the orchestrator might perform a **full deployment** of the application step by step, combining the primitives:

1. **Initialize Deployment:** The orchestrator gathers necessary input: target AWS region, environment name (e.g., "staging" or "prod"), any configuration specifics (instance sizes, number of instances, feature toggles), and the AWS credentials (for the target account) which it will soon encrypt. It might also generate unique names or identifiers for resources (like appending environment name to resource names) to avoid collisions.
2. **Credential Setup:** Orchestrator calls `GET /aws/public-key` on the AWS service to retrieve the encryption key. It then encrypts the AWS access key and secret. From this point on, all calls include the `credentials` field as the encrypted blob.
3. **S3 Bucket Creation:** Orchestrator calls `POST /aws/s3/bucket/create` for each S3 bucket needed. In our application, perhaps we need one bucket for application data, one for storing deployment packages, etc. It supplies the desired bucket names. The AWS service creates the buckets. The orchestrator might call the corresponding `GET /aws/s3/bucket/check` to verify each bucket was created successfully (or the create call itself could return a success flag). If a bucket already existed (perhaps from a previous run that was not cleaned up), the create might return an error or a special code; the orchestrator should handle that (possibly by logging a warning and continuing, since a pre-existing bucket might be acceptable if it's supposed to be idempotent).
4. **Lambda Functions Deployment (if any):** For each serverless function component of the app, orchestrator calls `POST /aws/lambda/create`. For example, if our application has an authentication service as a Lambda, and maybe a processing job as another Lambda, it would call the endpoint with the function details and code location. The AWS service creates the Lambda and (if requested) returns the function URL. The orchestrator can call `GET /aws/lambda/check` or possibly even invoke the function via its URL or AWS SDK to ensure it's responding. At minimum, confirming that the function was created (status is Active) is good. These Lambdas might operate independently (e.g., front-end might call them via the function URL, so just having the URL is needed) or they could be part of a bigger system. Save the returned URLs or ARNs for output or for other steps if needed.
5. **EC2 Instance and AMI (if needed):** If the deployment involves EC2 instances (for example, running a web server or database on EC2), the orchestrator needs an AMI. Two paths are possible:
6. **Use Pre-Baked AMI:** Perhaps we already have a pre-made AMI (golden image) for our application version. In that case, this step can be skipped or just record the AMI ID from configuration.

7. **Create/Bake AMI on the Fly:** If we need to create an AMI as part of deployment (for example, build a new image with the latest code), the orchestrator would launch a temporary EC2 instance first. It could do that by calling `POST /aws/ec2/instance/launch` with a base AMI and user data that installs the application. Once that instance is up and configured (the orchestrator might use the **execute command** endpoint to run installation scripts or verify installation on that instance), then orchestrator calls `POST /aws/ec2/ami/create` with the instance ID to snapshot it into a new AMI. After the AMI is created, the temporary instance can be terminated (the AWS service could have an endpoint for terminating an instance, or we can include that as part of this process). Now we have an AMI with the application ready to go. This approach aligns with “immutable infrastructure” principles where you bake everything into the image. It may take longer, but ensures consistency. The orchestrator will capture the new AMI ID from the response.
8. **Target Group and Load Balancer:** Orchestrator calls `POST /aws/alb/target-group/create` to set up a new target group for this deployment (if one doesn't already exist). For instance, target group name could be "myapp-web-prod-tg". The service returns the target group ARN. If we need an Application Load Balancer (ALB) and one isn't pre-existing, the orchestrator might also call `POST /aws/alb/create` to create a new ALB (with listeners). That would require specifying subnets, security groups, etc., and linking it to the target group (listeners forward traffic to the TG). For simplicity, we might assume the ALB is either created once and persists (with the target group attached each time) or is part of a separate network setup. We will clarify this in design; but we definitely create a target group because the ASG will need it.
9. **Auto Scaling Group Creation:** Orchestrator now calls `POST /aws/autoscaling/group/create` with parameters including the new AMI ID (from step 5), the target group ARN (from step 6), desired number of instances (e.g., 2 instances to start), min/max size, instance type, etc. The AWS service will create the Launch Template and ASG accordingly. The ASG will start launching EC2 instances (using the AMI). The call can return immediately after creation, but the instances will take some time to spin up. The orchestrator should then poll or wait until the instances are running and healthy. It can use a combination of `GET /aws/autoscaling/group/status` (to see if desired count is met and instances are in service) and possibly `GET /aws/alb/target-group/health` (to ensure the ALB target group reports the instances as healthy). This might take a few minutes, so the orchestrator might implement a retry loop with delays, or the AWS service could offer a blocking wait option (though usually it's better to keep it non-blocking at the service level and let the orchestrator handle waiting logic).
10. **Verification:** Once the ASG reports healthy instances (or the Lambdas are all deployed, etc.), the orchestrator can perform final verification. This might include: making a test HTTP request to the application's endpoint (which could be the ALB's DNS name or the Lambda URLs) to ensure the application is responding as expected. This is not strictly part of the AWS service API, but a higher-level check. Any issues here might trigger a rollback or re-deployment attempt depending on our strategy (rollback would be manual unless we script it). For now, the plan is at least to detect failures early.
11. **Output/Completion:** The orchestrator would then mark the deployment as successful and output relevant information (e.g., "Application deployed in us-east-1 - Load Balancer URL is X, Lambda URLs are Y and Z"). This marks the end of an environment spin-up.

Throughout this flow, if any step fails, the orchestrator can decide to perform a **teardown** (using the delete endpoints for any resources created so far) to avoid leaving partial infrastructure running. For example, if Lambda creation failed half-way, it might delete the buckets that were created earlier to clean up. Designing the orchestrator to handle partial failures will be important for reliability (though initial implementation might assume a clean environment or not handle rollback fully; this can be improved later).

A similar flow would be executed in reverse for **spinning down** an environment: the orchestrator would call the deletion endpoints roughly in the opposite order: delete ASG (which terminates instances), delete target group (and ALB if applicable), delete Lambdas, delete buckets, etc. This would free all allocated resources. We should ensure the AWS service's delete operations are robust (e.g., handle dependencies like an ASG must be deleted before its target group, etc., or have clear error messages if misordered so the orchestrator can adjust).

In summary, the build orchestrator is like the conductor, and the AWS service provides the instruments. This orchestration approach is powerful because we can program conditional logic, loops, and parallelism if needed in the orchestrator code, which is harder to do in declarative templates. It's essentially building a custom deployment tool tailored to our needs, while leveraging AWS's primitives.

Security Considerations

Security is a first-class concern in this project, given that we are handling cloud credentials and making potentially destructive changes (creating and deleting infrastructure). We have already touched on the **credential forwarding mechanism** using public key encryption to ensure the build orchestrator never sees raw secrets. Here we outline additional security considerations and how we plan to address them:

- **Least Privilege AWS Credentials:** The AWS credentials that we encrypt and pass to the AWS service should ideally belong to an IAM role or user with the minimal permissions required to perform the actions in our deployment. For example, if our deployment only needs to work with EC2, S3, Lambda, and Auto Scaling, the credentials should not have full admin rights to the account, but rather a policy that allows managing those specific services (and perhaps read-only for others if necessary). This limits the blast radius if the credentials were ever misused. We will work with the cloud admin to set up an IAM role for deployment automation with the required policies. The role's keys can be rotated regularly as well.
- **Secure Storage of Keys:** The private key for the AWS service (used to decrypt credentials) must be stored securely. Since the AWS service is likely stateless (could be running in a container or serverless function), we'll probably store the private key in a secure storage (like AWS Secrets Manager or an environment variable set through a secure pipeline) on the service side. It will be loaded at runtime to decrypt incoming data. Access to this key should be tightly controlled. If possible, using AWS KMS could be an alternative (where the build service uses KMS to encrypt credentials for the AWS service to decrypt), but that adds complexity – our own public/private key approach is fine and under our control.
- **Transport Security:** All API calls between the build orchestrator and the AWS service should be over HTTPS (TLS) to prevent any eavesdropping or man-in-the-middle attacks. Even though the credentials are encrypted at the payload level, we want to protect all other aspects of the communication (and it's standard practice to secure any web service). We will ensure the AWS service is hosted with TLS (perhaps behind an API gateway or load balancer that terminates TLS, or using an HTTPS-enabled function/service).
- **Authentication & Authorization of API Calls:** Currently, we assume the build orchestrator is a trusted caller to the AWS service (perhaps on the same network or with a known token). However, we should implement an auth mechanism for the AWS service API to prevent unauthorized usage. Options include: static API keys, mutual TLS, or a more complex OAuth/JWT if integrating with an identity service. A simple approach would be to generate an API token that the build service includes in headers of each call (in addition to the encrypted credentials payload). The AWS service would check this token. This prevents random actors from calling our service if they find the URL. Since this is internal, a static shared secret might suffice initially.
- **Auditing and Logging:** We will implement logging in the AWS service for each action (without sensitive data). For instance, logs should record "User X (or build orchestrator) called

CreateBucket for bucket Y in region Z – success” or any error messages from AWS. These logs are useful for debugging and for security auditing to know what was done in the AWS account and when. The build orchestrator should also log high-level progress of the deployment. If possible, we might integrate with CloudTrail on AWS side too (though since the actions use provided credentials, they will show up in CloudTrail under that IAM user’s activity). We must ensure logs do not accidentally log the plaintext credentials or the decrypted content. Logging the encrypted blob is okay (it’s gibberish to anyone without the key), but there’s not much need to log that either.

- **Input Validation:** Each API endpoint should validate inputs strictly. For example, ensure names conform to AWS requirements (bucket names must be globally unique and match a regex; Lambda names have length limits, etc.), and that required parameters are present. This prevents wasted calls to AWS that will error out and provides faster feedback. Also, by validating, we reduce the chance of something malicious or incorrect being passed through (though the service is internal, it’s good practice).
- **Resource Clean-up on Failures:** From a security and cost perspective, if a deployment fails mid-way, we should not leave resources running indefinitely. Part of security is ensuring we don’t unintentionally leave open S3 buckets or running EC2 instances that could be accessed. The orchestrator will be responsible for calling deletion endpoints if needed. We may enhance the AWS service to support tagging resources with a deployment ID, so that a cleanup script could later remove any stragglers (for example, tag everything with `DeploymentId=XYZ`; then a cleanup process could find all AWS resources with that tag and remove them if the deployment was aborted). This is more of a reliability concern but ties into security if something was public. For now, the manual or orchestrator-driven cleanup will be the approach.

By addressing these considerations, we aim to build the service such that it not only automates deployment but does so in a secure and controlled manner. We will also conduct a review (or even a threat model exercise) of the service before production use, to ensure no glaring security holes (e.g., injection flaws in our API, or leaving the private key accessible, etc.).

Implementation Plan and Steps

To realize this project, we will tackle it in phases. Below is a step-by-step implementation plan, along with key tasks and deliverables for each phase:

1. Design API Specification:

2. Define all endpoints of the AWS integration service in an API document. For each endpoint, specify the method (GET/POST/DELETE), URL path, expected request parameters (and format: JSON body or query params), and the response format. Also document error codes or conditions (e.g., what happens if a resource already exists, or if credentials are invalid). This spec will serve as a blueprint for implementation and for review by the team. We’ll circulate it for feedback to ensure it meets all requirements and is clear.

3. Set Up AWS Service Project:

4. Initialize a new code repository for the AWS service (if not already created). We will likely use Python (to leverage OSBot-AWS and possibly FastAPI or Flask to build the REST API). In fact, OSBot has a FastAPI integration (`osbot-fast-api` was noted ⁸, which might help scaffold the service quickly). We’ll set up the project structure, including configurations to run it (perhaps as a Docker container or a serverless function – to be decided if this service runs persistently or is invoked on demand).

5. Implement the basic server with a simple health-check endpoint (e.g., `GET /aws/health`) to verify the service is up. Also implement the `GET /aws/public-key` endpoint early, as it's simple and fundamental for subsequent testing.
6. Integrate the OSBot-AWS library by adding it as a dependency (`pip install osbot-aws`⁹). Ensure we can instantiate an AWS session using OSBot with dummy credentials to test connectivity (perhaps using environment credentials for now).

7. Implement Credential Encryption Mechanism:

8. Generate an RSA key pair for the service. This can be done either by pre-generating and storing it (e.g., check in an encrypted form or in environment variable) or programmatically on first run. We might generate it once and keep it constant so that the build service can cache the public key (regenerating on each start would require the build service to fetch a new key each time; that's not a problem actually, but if the service is ephemeral we might do that). We'll likely implement using a Python library like `cryptography` to handle RSA encryption/decryption.
9. Finalize the `GET /aws/public-key` endpoint to return the public key (in PEM format or perhaps as JSON Web Key). Also implement a utility function in the service to easily decrypt incoming data using the private key. Write unit tests for encryption/decryption (we can simulate the build service by encrypting a sample string with the public key and have the service decrypt it, verifying we get the original string). This ensures our mechanism works before moving on to real credentials.

10. Develop AWS Primitives Endpoints:

Implement each category of endpoints, one at a time, testing as we go:

11. **S3 Endpoints:** `bucket/create`, `bucket/check`, `bucket/delete`. Use OSBot-AWS's S3 helper class⁵¹⁰ to perform operations. Write tests that actually create a bucket (perhaps using a test AWS account or localstack). Ensure idempotency where possible (creating an existing bucket should either succeed no-op or return a clear error).
12. **Lambda Endpoints:** `lambda/create` (including function URL logic), `lambda/check`, `lambda/delete`. Use OSBot-AWS's Lambda class⁶⁷. This one will involve uploading or referencing code; for testing, we might use a simple hello-world deployment package. Verify that function URL creation works by doing a test invocation of the URL (maybe as part of integration testing).
13. **EC2/AMI Endpoints:** `instance/launch`, `instance/terminate` (for cleanup), `instance/run-command`, `ami/create`. OSBot-AWS likely has EC2 wrappers (we saw EC2 is supported¹¹). If not directly, we can use boto3 via OSBot's session. We need to carefully implement run-command (likely using AWS SSM SendCommand API – ensure the IAM role for the instance has SSM permissions). Test by launching a small instance (perhaps an EC2 micro with a known AMI like Amazon Linux) and running a harmless command (`echo hello`). This might be tricky to test in automated way, but we can at least verify the API call structure. AMI creation can be tested by creating an image of that running instance and then comparing that the AMI shows up in the account. Since creating AMIs and instances costs money and time, we might use these tests sparingly or rely on integration testing in a sandbox environment.
14. **Target Group & ASG Endpoints:** `target-group/create`, `alb/create` (if needed), `autoscaling/create`, and their checks. These are a bit more complex because they involve more moving parts. We can use boto3 through OSBot's session for these if OSBot doesn't have high-level methods. For example, boto3's `create_auto_scaling_group` and `create_launch_template` calls. We need to also possibly create an IAM role for EC2 if not

already (instances might need an IAM role for SSM or other). However, setting up the ASG fully might be beyond a simple unit test; we'll likely rely on careful code review and one end-to-end run in AWS to confirm these. If needed, we can break down the problem: first manually do it via AWS console or CLI to identify required parameters, then encode those in our service.

For each set of endpoints implemented, write **unit tests** using mocked AWS calls or a localstack (which OSBot supports for S3, and maybe for some other services). This will ensure our code is correct. Then perform a **dry-run** integration test in an actual AWS test account to validate things in reality. We expect some tweaking will be needed especially for ASG/ALB which are often fiddled with on real AWS.

1. Build Orchestrator Integration:

2. Update the build orchestrator code to use the new AWS service. This involves adding functions or modules that know how to call the AWS service's API (likely via HTTP calls). If the orchestrator is written in Python as well, we might incorporate a small client or just use `requests` to call the endpoints. We'll implement helper methods like `deploy_environment(env_name, region, aws_creds)` which internally sequences the calls as described in the Orchestration Flow section.
3. Migrate any existing manual steps or scripts into orchestrator logic using the new endpoints. For example, if currently deploying a Lambda was done by a manual CLI command, replace that with the orchestrator calling the `lambda/create` API. Essentially, encode the voice memo's described steps into code now that the service supports them.
4. Ensure that the orchestrator properly handles errors from the AWS service. Implement retry logic for transient failures (e.g., if create bucket fails due to a naming collision, maybe choose a new name or fail gracefully; if an AWS rate limit is hit, wait and retry). Also, implement the teardown logic in the orchestrator in case a failure occurs mid-way (to avoid leftover resources). We may also add a command or function in orchestrator specifically to destroy a named environment (which calls all the delete endpoints).

5. Testing End-to-End:

6. Perform a full **end-to-end test** of deploying an environment using the orchestrator and AWS service together. This means running the orchestrator (with appropriate configuration and credentials) to deploy, say, a **dev environment** in a test AWS account. Monitor the process, verify each step succeeds and that the application comes up reachable. This will likely reveal any issues in the flow (perhaps missing permissions, or a step we overlooked such as configuring security groups for the instances so the ALB can connect, etc.). We will iteratively fix any problems discovered.
7. Test the teardown as well: use the orchestrator to tear down the environment and verify all resources are indeed removed (S3 buckets gone, instances terminated and AMI deregistered if we choose to, etc.). Some resources like AMIs don't delete automatically (you have to deregister them and delete associated snapshots), so consider if we need to implement that as well. For now, we might leave AMIs in place (as they could be re-used), but perhaps tag them for easier manual cleaning.
8. If possible, also test an alternative mode (for example, a purely serverless deployment scenario versus the EC2 one) to ensure the orchestrator can branch and the AWS service endpoints cover both.

9. Deployment of AWS Service:

10. Decide how and where the AWS service will run in production. Options include: as a containerized service in our cluster, as a standalone EC2, or even as an AWS Lambda (if the service itself could be serverless). The choice depends on how the orchestrator can reach it and the expected load. It will probably not receive heavy traffic (only when deployments happen, which might be a few times a day), so a small container or a Lambda behind API Gateway could work. We'll likely deploy it as a Docker container in our existing infrastructure for simplicity.
11. Set up CI/CD for the AWS service: automated tests on commit, and deployment pipeline to push the service to the environment. Store the necessary configurations (like the private key, or generate it on deploy).
12. Once deployed, do a final sanity check: have the orchestrator call the live service to do a trivial action (maybe create then delete a dummy bucket) to ensure connectivity and permissions are correct in the production setup.
13. **Documentation & Training:**
14. Although this document itself is part of the documentation, we will write a user guide (for internal use) on how to trigger deployments using the build orchestrator (e.g., what command or script to run, what parameters to pass). Also document how to configure new target environments (for instance, if we want to deploy to a new AWS account, what setup (like creating an IAM role/keys and adding them to our secure store) is needed).
15. Document the AWS service API for future maintainers, possibly in a README in its repo or an internal wiki, so developers know what endpoints exist and what they do. This overlaps with the API spec but also covers how to extend it if needed to support new resource types.
16. Conduct a walkthrough with the team to explain the system, especially the security model. This will help everyone trust and understand the new process, and also gather any feedback or concerns.
17. **Further Improvements (Post-MVP):**
18. After the initial implementation is stable, consider adding enhancements. For example, implementing **rollback logic**: if a deployment fails, automatically trigger the teardown (this could even be built into the orchestrator as a wrapper that tries deployment and catches errors to then call destroy). Another improvement could be more sophisticated **status monitoring**, like a dashboard showing current deployments or their status (though that might be overkill; even logs might suffice).
19. We might also integrate notifications (e.g., Slack or email alerts when a deployment completes or fails). This can be done in the orchestrator since it has the high-level view.
20. In terms of the AWS service, future expansions could include supporting more AWS services if needed (for example, RDS database instances, CloudWatch alarms, etc., depending on what our application needs). The design is modular, so adding a new primitive like "Create RDS instance" would be straightforward if the need arises.

By following this implementation plan, we ensure a structured approach to building out the AWS deployment service and integrating it with our existing pipeline. The result will be a robust system where deploying or destroying an environment is as simple as running a command, with the heavy lifting done by our code rather than manually clicking in AWS – improving speed, consistency, and reliability of our deployments.

Conclusion

In this project, we outlined a detailed plan to build an AWS integration service that works hand-in-hand with our build orchestrator to automate cloud deployments. We started by understanding the context and motivation – replacing manual AWS setup with an automated process controlled by our build service, while maintaining a high standard of security (no plaintext credentials exposure) and flexibility (deploying to any region or environment on demand).

The proposed solution leverages the OSBot-AWS library's simplified AWS APIs ³ ⁴ to implement a suite of REST endpoints (primitives) for operations like creating S3 buckets, launching EC2 instances, configuring Lambda functions, and setting up auto-scaling groups. Each of these operations can be invoked independently, allowing our orchestrator to mix-and-match them to realize the full infrastructure deployment. We drew parallels to Terraform/CloudFormation for the concept of infrastructure-as-code ¹ ², but our design chooses an imperative approach with fine-grained control. This means we can, for example, encrypt credentials per call and handle custom logic between steps, which is harder to do with purely declarative templates.

By implementing this plan, we expect to achieve: faster environment provisioning (potentially going from hours of manual setup to minutes of automated work), more consistent environments (since the same automated steps run every time), and easier multi-region deployments (just by changing parameters). Moreover, the investment in this automation will pay off in the long run by enabling continuous delivery practices – we could spin up ephemeral test environments for each feature branch or run integration tests on a fresh stack and then tear it down, all automatically.

This project plan serves as a roadmap. The next action is to proceed with the implementation steps, starting with finalizing the API spec and then coding the AWS service endpoints. Regular reviews will be conducted to ensure alignment with the objectives. With careful execution, this AWS deployment service will become a cornerstone of our deployment pipeline, providing a secure and powerful way to manage our cloud infrastructure.

¹ Terraform | HashiCorp Developer

<https://developer.hashicorp.com/terraform>

² AWS CloudFormation: A Comprehensive Guide to Infrastructure Automation

<https://www.cloudoptimo.com/blog/aws-cloudformation-a-comprehensive-guide-to-infrastructure-automation/>

³ ⁴ ⁵ ⁶ ⁷ ⁹ ¹⁰ ¹¹ GitHub - owasp-sbot/OSBot-AWS: AWS apis and commands for the OWASP Security Bot (OSBot)

<https://github.com/owasp-sbot/OSBot-AWS>

⁸ osbot-fast-api - PyPI

<https://pypi.org/project/osbot-fast-api/>